

# **Introduction to Amazon API Gateway: Hands-on Lab**

## Table of Contents

|                                                                      |                 |
|----------------------------------------------------------------------|-----------------|
| <b><u>INTRODUCTION TO AMAZON API GATEWAY: HANDS-ON LAB .....</u></b> | <b><u>1</u></b> |
| <b>LAB OVERVIEW .....</b>                                            | <b>3</b>        |
| <b>OBJECTIVES .....</b>                                              | <b>3</b>        |
| <b>LAB ENVIRONMENT .....</b>                                         | <b>3</b>        |
| <b>AMAZON API GATEWAY AND AWS LAMBDA.....</b>                        | <b>3</b>        |
| <b>AWS LAMBDA .....</b>                                              | <b>4</b>        |
| <b>TASK 1: CREATE A LAMBDA FUNCTION.....</b>                         | <b>5</b>        |
| <b>TASK 1.1: CREATE THE INITIAL LAMBDA FUNCTION .....</b>            | <b>5</b>        |
| <b>TASK 2: TEST THE LAMBDA FUNCTION .....</b>                        | <b>9</b>        |

## Lab overview

In this lab, I created a simple FAQ micro-service. The micro-service returns a JSON object containing a random question and answer pair using an **Amazon API Gateway** endpoint that invokes an **AWS Lambda** function.

The user initiates a request, which can be an HTTP or GET request, targeting services hosted within the AWS cloud. This request first reaches Amazon API Gateway, which is responsible for transforming the HTTP request into a JSON format. Subsequently, this formatted JSON is sent through a VPC endpoint to an AWS Lambda function.

The Lambda function processes the incoming request and generates a response, also in JSON format. This response is then sent back through the VPC endpoint to Amazon API Gateway, which converts it into a final HTTP response that is returned to the user.

## Objectives

By the end of this lab, I was able to accomplish the following:

- create an **AWS Lambda function**,
- set up an **Amazon API Gateway** endpoint,
- and debug both **API Gateway** and **Lambda** using Amazon CloudWatch.

## Lab environment

### Amazon API Gateway and AWS Lambda

A microservice that uses Amazon API Gateway includes a defined resource and associated methods (such as GET, POST, PUT, etc.) within API Gateway, as well as a backend target (AWS, n.d).

In this lab, the backend target is an AWS Lambda function. However, the backend target can also be any other HTTP endpoint (like a third-party API or a web server), an AWS service proxy, or a mock integration used as a placeholder (AWS, n.d).

## Services Used in This Lab

### Amazon API Gateway

Amazon API Gateway is a managed service offered by AWS that simplifies the creation, deployment, and maintenance of APIs. It includes various features to enhance API functionality:

- **Request and Response Transformation:** Modify the body and headers of incoming API requests to align with backend systems, and similarly adjust the body and headers of outgoing API responses to meet API requirements.
- **Access Control:** Manage API access using AWS Identity and Access Management.
- **API Key Management:** Create and implement API keys for third-party developers.
- **Monitoring:** Integrate with Amazon CloudWatch to monitor API performance.
- **Caching:** Utilize Amazon CloudFront to cache API responses for improved response times.
- **Multi-Stage Deployment:** Deploy an API to multiple stages, facilitating differentiation between development, testing, production, and versioning.
- **Custom Domain Connection:** Connect custom domains to your API.
- **Model Definition:** Define models to standardize transformations of API requests and responses.

### AWS Lambda

Is a powerful serverless, event-driven compute service that enables you to execute code without the need for server management. It works seamlessly with over 200 AWS services and third-party applications, allowing you to trigger functions through various events like user actions on an e-commerce site. This flexibility means you only pay for the compute time you actually use.

With AWS Lambda, you can enhance existing AWS services with custom logic, create scalable backend solutions, and automate administrative tasks. The service supports a range of features including built-in fault tolerance, automatic scaling, and the ability to package functions as container images.

You can also connect Lambda functions to relational databases and shared file systems, orchestrate multiple functions, and integrate it with AWS Identity and Access Management (IAM) for a secure environment. Plus, monitoring and observability tools help you keep track of your applications effectively.

## Task 1: Create a Lambda Function

In this task, I used the AWS Lambda Console to create a function with lab provided code, and configured it to work with Amazon API Gateway.

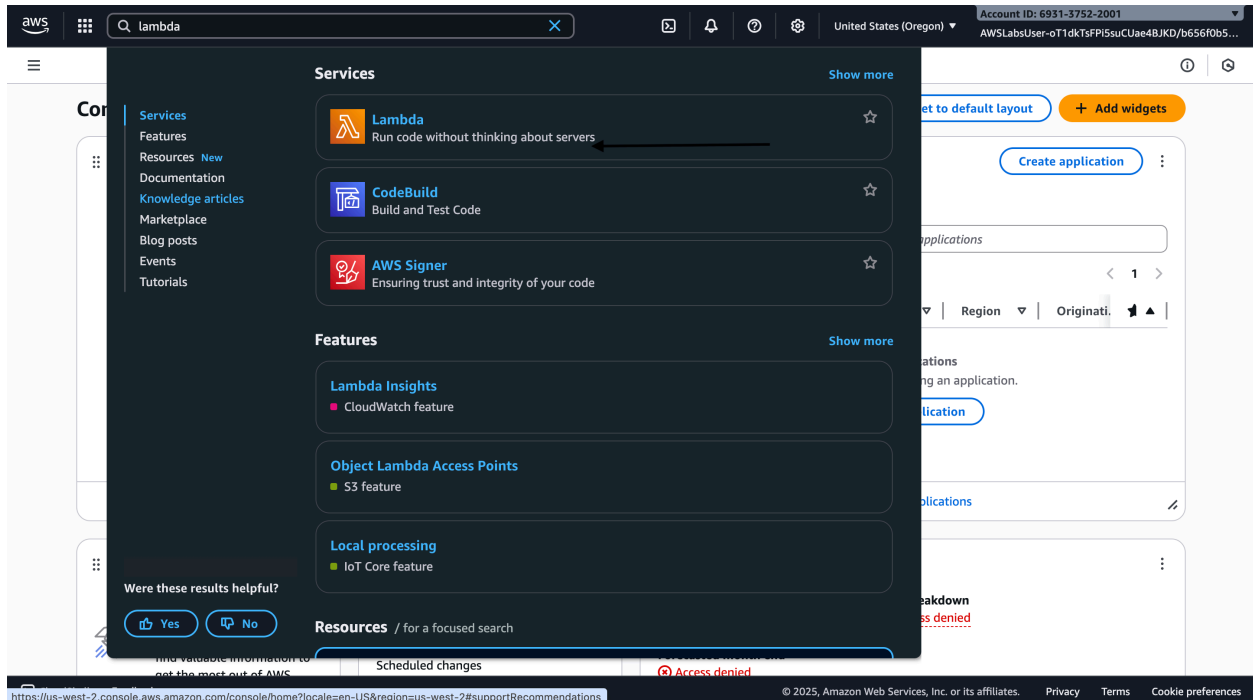


Figure 1: Lambda console (source: personal collection)

### Task 1.1: Create the initial Lambda Function

- Step 1: Select “Author from scratch”
- Step 2: set “Function name” to FAQ
- Step 3: set “Runtime” to “Node.js 18.x”
- Step 4: “Change default execution role” |
- Step 5: “Use an existing role” inside “Execution role”
- Step 6: “choose VPC”
- Step 7: select VPC with CIDR range 10.0.0.0/16
- Step 8: select two subnets with CIDR: 10.0.1.0/24, 10.0.2.0/24
- Step 9: “Create function”

**Create function** [info](#)

Choose one of the following options to create your function.

- ☒ **Author from scratch**  
Start with a simple Hello World example.
- ☐ **Use a blueprint**  
Build a Lambda application from sample code and configuration presets for common use cases.
- ☐ **Container image**  
Select a container image to deploy for your function.

**Basic information**

**Function name**  
Enter a name that describes the purpose of your function.  
FAQ  
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (\_).

**Runtime** [info](#)  
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.  
Node.js 20.x

**Architecture** [info](#)  
Choose the instruction set architecture you want for your function code.  
☐ arm64  
☒ x86\_64

**Permissions** [info](#)  
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

Execution role

Figure 2: Create function page (source: personal collection)

**Permissions** [info](#)  
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

▼ **Change default execution role**

**Execution role**  
Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).  
☐ Create a new role with basic Lambda permissions  
☒ Use an existing role  
☐ Create a new role from AWS policy templates

**Existing role**  
Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.  
lambda-basic-execution  
[View the lambda-basic-execution role](#) on the IAM console.

▼ **Additional configurations**  
Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

**Networking**

**Function URL** [info](#)  
Use function URLs to assign HTTP(S) endpoints to your Lambda function.  
☐ Enable

**VPC** [info](#)  
Connect your function to a VPC to access private resources during invocation.  
☒ Enable

**VPC** [info](#)  
Select a VPC for your resource to access.  
vpc-0F18aceb06f5f46d7 (10.0.0.0/16)  
☐ Allow IPv6 traffic for dual-stack subnets  
You can allow outbound IPv6 traffic to subnets that have both IPv4 and IPv6 CIDR blocks.

Figure 3: Additional Configuration tabs (source: personal collection)

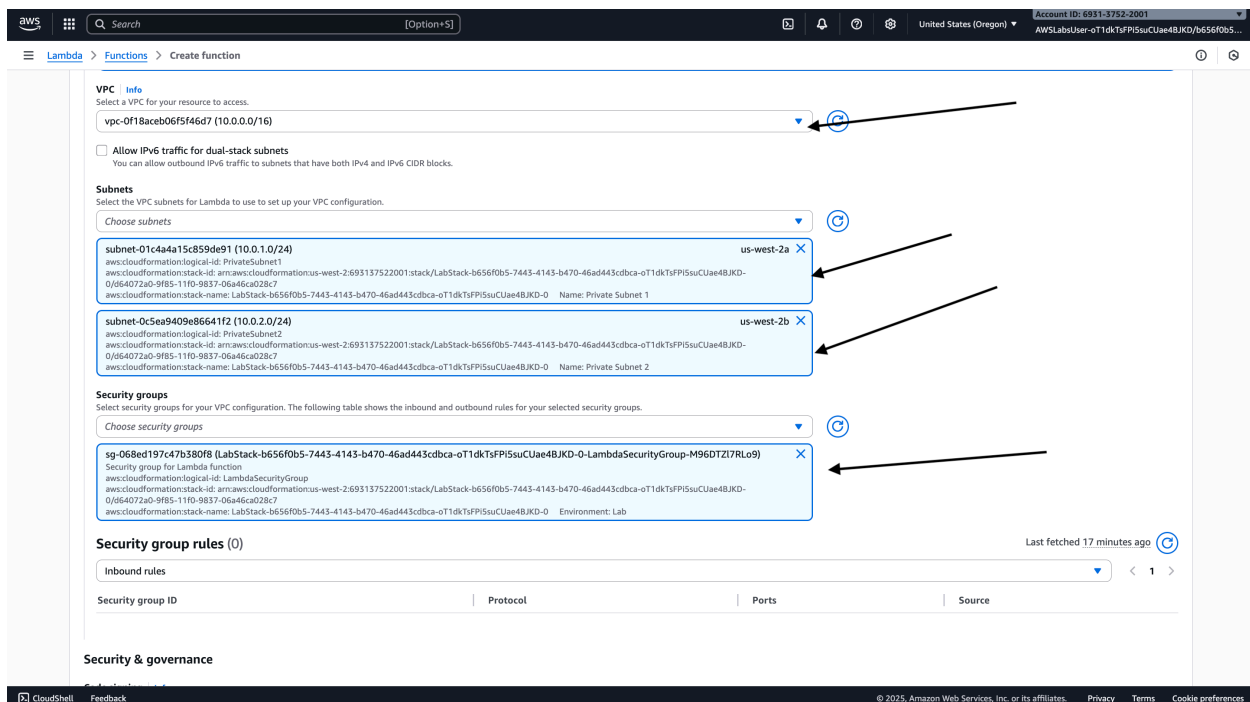


Figure 4: Subnets & Security groups configuration (source: personal collection)

- Step 10: set up the JSON codes
- Step 11: deploy the code

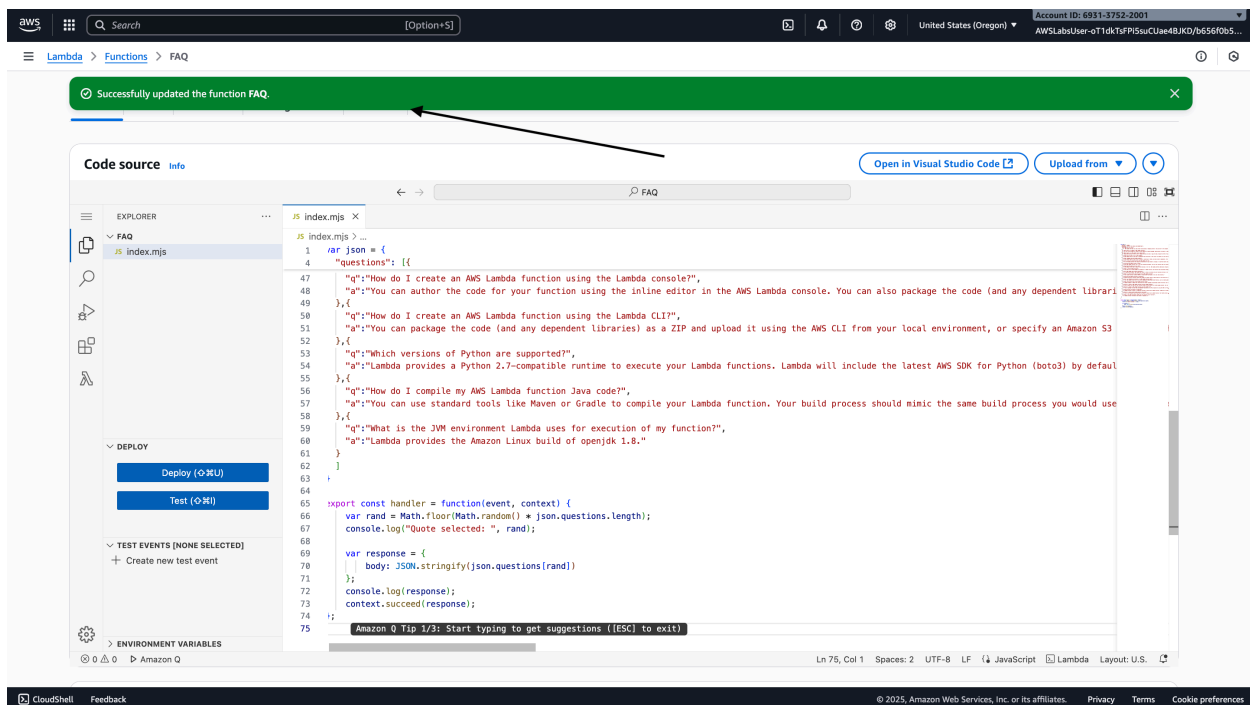


Figure 5: Deployment success (source: personal collection)

## Task 2: Create an API Gateway endpoint

An API endpoint is a hostname of the API. This can be edge-optimized or regional, depending on where the traffic originates from.

- Step 1: choose the “Configuration” tab
- Step 2: choose “General configuration”, then “edit”
- Step 3: select “Description”
- Step 4: “save”

The screenshot shows the AWS Lambda console's 'Edit basic settings' page. The 'Basic settings' section is active, showing a 'Description' field with the text 'Provide a random FAQ'. Below this, the 'Memory' section is set to 128 MB, 'Ephemeral storage' is set to 512 MB, and 'SnapStart' is set to 'None'. The 'Timeout' is set to 0 minutes and 3 seconds. Under 'Execution role', the 'Use an existing role' option is selected, and the role 'lambda-basic-execution' is chosen. At the bottom right of the configuration panel, there are 'Cancel' and 'Save' buttons. A red arrow points to the 'Save' button.

Figure 6: General Configuration tab (source: personal collection)

- Step 5: select “API Gateway” as the source
- Step 6: select “Create a new API”
- Step 7: select “REST API” as “API type”
- Step 8: select “API name”
- Step 9: set “Deployment stage”
- Step 10: choose “add”



**Trigger configuration** [Info](#)

**API Gateway**  
aws api application-services backend HTTP REST serverless

Add an API to your Lambda function to create an HTTP endpoint that invokes your function. API Gateway supports REST, HTTP, and WebSocket APIs. [Learn more](#)

**Intent**  
Use an existing api or have us create one for you.  
☒ Create a new API  
☐ Use existing API

**API type**  
☐ HTTP API  
☐ WebSocket API  
☒ REST API

**Security**  
Configure the security mechanism for your API endpoint.  
Open

**Additional settings**

**API name**  
Choose a name for your API. API names don't need to be unique.  
FAQ-API

**Deployment stage**  
Select the API stage where Lambda adds the trigger. You can use the deployment history in the API Gateway console to revert or change the active deployment of this stage.  
myDeployment

☐ Enable metrics and error logging  
Record latency and error metrics in Amazon CloudWatch, and log errors in Amazon CloudWatch Logs. Standard CloudWatch and CloudWatch Logs pricing applies.

**Binary media types**  
Specify response types that API Gateway should treat as binary data instead of text. To treat all responses as binary data, use \*/\*. If you enter one or more specific types, you must also set the Content-Type header in the response object that you return from your function code. [Learn more](#)

Figure 7: Add trigger page (source: personal collection)

## Task 2: Test the Lambda function

- Step 1: select “Triggers” from the navigation menu on the left
- Step 2: select “API Gateway” and expand the “details”
- Step 3: open the “API endpoint” on the browser

**Successfully updated the function FAQ.**

**FAQ**  
Layers (0)

[+ Add trigger](#) [+ Add destination](#)

**Description**  
Provide a random FAQ

**Last modified**  
10 seconds ago

**Function ARN**  
arn:aws:lambda:us-west-2:693137522001:function:FAQ

**Function URL** [Info](#)

**Configuration** [Info](#)

**Description**  
Provide a random FAQ

**Timeout**  
0 min 3 sec

**Memory**  
128 MB

**Ephemeral storage**  
512 MB

**SnapStart**  
None

[Edit](#)

aws

Search

[Option+5]

United States (Oregon)

Account ID: 6931-3752-2001

AWS::IAM::User-T1dkTafP3ouC1ue4BjKQ/b656f0b5...

Lambda

Functions

FAQ

FAQ

Throttle

Copy ARN

Actions

The trigger FAQ-API was successfully added to function FAQ. The function is now receiving events from the trigger.

Function overview

Diagram

Template

FAQ

Layers

(0)

API Gateway

+ Add trigger

+ Add destination

Description

Provide a random FAQ

Last modified

4 minutes ago

Function ARN

arn:aws:lambda:us-west-2:693137522001:function:FAQ

Function URL

Info

Code

Test

Monitor

Configuration

Aliases

Versions

General configuration

Triggers

Permissions

Destinations

Function URL

Environment variables

Tags

VPC

General configuration

Description

Provide a random FAQ

Memory

128 MB

Timeout

0 min 3 sec

SnapStart

None

Info

Ephemeral storage

512 MB

Edit

The screenshot shows the AWS Lambda console interface. At the top, the navigation bar includes the AWS logo, a search bar, and account information (Account ID: 6931-3752-2001, United States (Oregon)). The main header shows the path: Lambda > Functions > FAQ. Below this, there are buttons for '+ Add trigger', '+ Add destination', and 'arn:aws:lambda:us-west-2:693137522001:function:FAQ'. The 'Function URL' is also displayed with an 'Info' link.

The 'Configuration' tab is selected, showing a sidebar with various configuration options: General configuration, Triggers, Permissions, Destinations, Function URL, Environment variables, Tags, VPC, RDS databases, Monitoring and operations tools, Concurrency and recursion detection, Asynchronous invocation, Code signing, File systems, and State machines. An arrow points to the 'Triggers' option in the sidebar.

The 'Triggers (1) Info' section displays a list of triggers. A single trigger is shown: 'API Gateway: FAQ-API'. An arrow points to this trigger. The details for this trigger are as follows:

- API endpoint: <https://gsflp3kr2.execute-api.us-west-2.amazonaws.com/myDeployment/FAQ>
- ▼ Details
- API type: REST
- Authorization: NONE
- isComplexStatement: No
- Method: ANY
- Resource path: /FAQ
- Service principal: apigateway.amazonaws.com
- Stage: myDeployment
- Statement ID: lambda-76c31634-e040-445f-8278-46ad07e567e9

At the bottom of the console, there is a footer with 'CloudShell', 'Feedback', and copyright information: '© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences'.

Figure 10: Triggers (source: personal collection)

- Step 4: Configure a new test event “BasicTest”
- Step 5: keep the JSON object empty
- Step 6: Save
- Step 7: Test
- Step 8: verify the execution result’s details
- Step 9: Access the “Monitor” tab
- Step 10: choose “Cloudwatch logs” and analyze the log streams

The screenshot displays the AWS Lambda console interface. At the top, a green notification bar states: "The test event 'BasicTest' was successfully saved." Below this, the "Monitoring" tab is active, showing the execution details for a function named "BasicTest".

**Executing function: succeeded** (logs [logs](#))

**Details**

```
{
  "body": "[\"g\": \"Which versions of Python are supported?\", \"a\": \"Lambda provides a Python 2.7-compatible runtime to execute your Lambda functions. Lambda will include the latest AWS SDK for Python (boto3) by default.\"]"
}
```

**Summary**

|                                                                    |                                                           |
|--------------------------------------------------------------------|-----------------------------------------------------------|
| <b>Code SHA-256</b><br>0yeEEVxDZL1DzGvDQ0rRfPjWYwOGN5VszXTps5deb4= | <b>Execution time</b><br>22 seconds ago                   |
| <b>Function version</b><br>SLATEST                                 | <b>Request ID</b><br>fedb39ce-dabc-4ff4-98ea-4b88377f86ff |
| <b>Duration</b><br>64.61 ms                                        | <b>Billed duration</b><br>209 ms                          |
| <b>Resources configured</b><br>128 MB                              | <b>Max memory used</b><br>68 MB                           |
| <b>Init duration</b><br>143.84 ms                                  |                                                           |

**Log output**

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

```
START RequestId: fedb39ce-dabc-4ff4-98ea-4b88377f86ff Version: SLATEST
2025-10-02T13:50:00.656Z    fedb39ce-dabc-4ff4-98ea-4b88377f86ff    INFO    Quote selected: 16
2025-10-02T13:50:00.661Z    fedb39ce-dabc-4ff4-98ea-4b88377f86ff    INFO    {
  body: "[\"g\": \"Which versions of Python are supported?\", \"a\": \"Lambda provides a Python 2.7-compatible runtime to execute your Lambda functions. Lambda will include the latest AWS SDK for Python (boto3) by default.\"]"
}
END RequestId: fedb39ce-dabc-4ff4-98ea-4b88377f86ff
REPORT RequestId: fedb39ce-dabc-4ff4-98ea-4b88377f86ff  Duration: 64.61 ms    Billed Duration: 209 ms Memory Size: 128 MB    Max Memory Used: 68 MB Init Duration: 143.84 ms
```

The footer of the console shows "CloudShell", "Feedback", and copyright information for Amazon Web Services, Inc. or its affiliates, along with links for "Privacy", "Terms", and "Cookie preferences".

Figure 11: Test results (source: personal collection)

The screenshot shows the AWS CloudWatch console interface. The top navigation bar includes the AWS logo, a search bar, and account information. The left sidebar contains navigation links for CloudWatch, Log groups, and various monitoring tools. The main content area is titled 'Log events' and shows a list of log entries for a specific Lambda function. The log entries include timestamps and messages detailing the function's execution, such as 'INIT\_START', 'START', and 'END' events.

| Timestamp                     | Message                                                                                                                                                              |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2025-10-02T16:50:00.507+03:00 | INIT_START Runtime Version: nodejs:20.v79 Runtime Version ARN: arn:aws:lambda:us-west-2::runtime:01eab27397bfdbdc243d363698d4bc355e3c8176d34a51c47dfe58cf977f508     |
| 2025-10-02T16:50:00.654+03:00 | START RequestId: fedb39ce-dabc-4ff4-98ea-4b88377f86ff Version: \$LATEST                                                                                              |
| 2025-10-02T16:50:00.656+03:00 | 2025-10-02T13:50:00.656Z fedb39ce-dabc-4ff4-98ea-4b88377f86ff INFO Quote selected: 16                                                                                |
| 2025-10-02T16:50:00.661+03:00 | 2025-10-02T13:50:00.661Z fedb39ce-dabc-4ff4-98ea-4b88377f86ff INFO { body: '{"q": "Which versions of Python are supported?", "a": "Lambda provides a Python 2.7-co.. |
| 2025-10-02T16:50:00.720+03:00 | END RequestId: fedb39ce-dabc-4ff4-98ea-4b88377f86ff                                                                                                                  |
| 2025-10-02T16:50:00.720+03:00 | REPORT RequestId: fedb39ce-dabc-4ff4-98ea-4b88377f86ff Duration: 64.61 ms Billed Duration: 209 ms Memory Size: 128 MB Max Memory Used: 68 MB Init Duration: 143..    |

Figure 12: Cloudwatch page (source: personal collection)

## Conclusion

In this lab, I explored how to create a microservice using a Lambda function, then connect it to the API Gateway endpoints, and finally Debug API Gateway and Lambda with Amazon CloudWatch.

## Reference

Amazon Web Services. (n.d.). *Introduction to Amazon API Gateway*. Retrieved from <https://lab.builder-labs.skillbuilder.aws/sa/lab/arn%3Aaws%3Alearningcontent%3Aus-east-1%3A470679935125%3Ablueprintversion%2Fspl-58%3A2.0.34-68467076/en-US/b656f0b5-7443-4143-b470-46ad443cdbca::oT1dkTsFPi5suCUae4BJKD>